Reproducing “Near-Duplicate Detection for Web Crawling”

Name Surname

2026-08-01

Abstract This paper aims to reproduce the findings of the “Near-Duplicate Detection for Web Crawling” work, using PySpark to scale the proposed algorithm, under different datasets, feature models and parameters.

Introduction

This work aims to answer the following question: given two datasets, is it possible to detect near duplicate pairs of data efficiently?

That question was tackled by Manku et al.'s paper [1], applied to web crawling.

The “Definitions and Terminology” chapter formalizes and explains the terms used throughout this paper, while the “Methodology” chapter contains the higher-level logic of the solution proposed. “Implementation” chapter talks about implementation details and lastly “Result” shows an application of the implementation across multiple datasets.

Definitions and Terminology

This paragraphs includes common terminology used in later chapters. In this paper only textual data (i.e. strings) are considered, however the approach is similar to any other type of data.

- Two data are *duplicate* if they are exactly equal. In practice, their hash digest is the same.
- Two data are *near-duplicate* if they are *similar enough* to some criteria.
- A *feature* is a property of some data, obtained from the application of a *feature model*. For example, a feature value can be 32 and its feature model might be “the number of words in the string”. Note that a single data might have multiple features.
- A *fingerprint* is a binary string of fixed length that represents the content of some data.
- A *partition* of a fingerprint is a substring of that fingerprint. Each partition is exclusive: it cannot overlap with other partitions.
- A *schema* defines how many partitions should be used and how to combine them for the near-duplicate algorithm.

Methodology

Consider an easier variant of the tracted problem: given two datasets, is it possible to detect ~~near~~ duplicate pairs of data ~~efficiently~~?

This can be trivially solved computing hash digests for every data in the datasets and using a Cartesian product: if two digests match then they are duplicate (ignoring hash collisions).

However, this wouldn't work to detect *near* duplicates: two almost identical data might have wildly different digests. Also, a Cartesian product is too inefficient to process large datasets.

The following sub-chapters aim to solve these issues, replicating the original paper [1] methodology.

1. Fingerprint Generation

As previously mentioned, hash functions can't represent data for near-duplicate detection purposes.

In order to solve this issue, Charikar's SimHash [2] algorithm is used. Given an input string and a feature model, its approach can be described as follows: 1. Features are extracted from the input; 2. All features are hashed; 3. The features digests are combined into a fingerprint, taking the most popular bit value for each bit index, across all digests.

Selecting the right feature model is crucial: too many features slows down SimHash, while too little will drop performance down. In the following subchapters, four different feature models are considered.

Bag of Words

For a given text, its features are the individual words it contains. Hence, two texts are considered near-duplicate if they contain about the same words. This approach is computationally efficient and simple to implement, but it fails to preserve spatial locality. For example, the sentence "The bank is after the office" yields the exact same features as "The office is after the bank".

Shingles

Generally, a shingle is a contiguous sequence of n items from a data source; in this context, it refers to an overlapping contiguous sequence of n characters. Shingles are generated using a sliding window algorithm with a window size of n. This model preserves locality to some extent, but it generates a large number of features, which can negatively impact performance. To reduce the feature count, a non-overlapping variant is considered. Additionally, word-based shingles (as opposed to traditional character-based ones) are evaluated.

TF-IDF

A refined version of *Bag of Words* that accounts for both the frequency of a word within a specific text (DF) and its rarity across the entire dataset (IDF):

$$\text{TF-IDF}(term, document, Dataset) = TF(term, document) * IDF(term, Dataset)$$

TF stands for Term Frequency, which assigns higher importance to words that appear frequently in a given text:

$$TF(term, document) = \frac{\text{How many times term appears in document}}{\text{How many words document has}}$$

IDF stands for Inverse Document Frequency, which assigns higher weight to words that appear in fewer documents. This filters out very common words, such as conjunctions and verbs:

$$IDF(term, Dataset) = \log(\frac{\text{Total number of documents in Dataset}}{\text{Number of documents that contain term}})$$

This is the feature model chosen in the original paper [1].

Despite its higher accuracy, it's computationally more expensive than previous models, as it requires dataset-wide transformations. Furthermore, maintenance introduces additional overhead, as the weights of each term must be recomputed whenever the dataset changes.

Embeddings
An embedding model is a machine learning algorithm that maps text into a vector capturing its semantic meaning. In this study, a transformer-based model, *Qwen3-embedding-0.6b* [3], is used and executed locally using Ollama. Each weight of the generated embeddings is quantized to 0/1, taking the median of all weights as pivot, becoming a bit of the result fingerprint. Although suboptimal, this is required in order to build binary fingerprints. While all previous models rely on syntax to extract features, embedding models operate on semantics, allowing for the detection of semantical near-duplicate text.

2. Fingerprint Matching

All data now has an associated fingerprint.

The goal for this phase is to compare fingerprints from two datasets, called *dataset* and *query* for simplicity sake, in order to find pairs of fingerprints whose Hamming distance is less than K, a fixed integer value.

The reference paper [1] approaches this problem by splitting each fingerprint into at least $K + n$ *partitions*, for $n > 0$. This grants a useful property: if two fingerprints are near-duplicate by a factor of K, then at least n of their $K + n$ partitions must be equal (the opposite doesn't hold - other partitions might have more than K different bits). The optimal schema is derived by the following formula:

$$X(f, k, d) = \begin{cases} 1 & \text{if } \ d < \tau \\ \min_{r>k} \binom{r}{k} \cdot X\left(\frac{fk}{r}, k, d - \frac{(r-k)f}{r}\right) & \text{otherwise} \end{cases}$$

where f is the size of each fingerprint, K is the maximum allowed Hamming distance, d is the log_2 size of query and $p_m in$ is the maximum allowed candidate set size (in log_2).

As previously stated, for a given K value it's necessary to create at least $K + n$ partitions, for any $n > 0$. As K grows, the number of partitions also grows, shrinking the partition size and exploding the number of candidates for each partition. Image [image-K-complexity] shows that, for a fingerprint size of 64 bits, a K value below 6 is suggested (approximately 10% of the fingerprint size).

Manku's approach can be summerized as follows:

1. n copies of *query* are generated, where $n = \binom{r}{k}$, computed by the previous formula
2. For each copy, a schema instance is applied: a partition of all fingerprints of a copy are cut and pasted on the head of all fingerprints
3. Fingerprints are sorted and then compressed, using Huffman codes
4. All query copies are broadcasted to workers
5. Workers iterate over *dataset*, extracting partitions from all rows and checking if it matches at least one of the *query* fingerprints (with the same schema instance), using binary search
6. Fingerprint pairs with matching partitions *might* be near-duplicate: if their Hamming distance is less than K they are considered a near-duplicate pair

Implementation

This implementation uses the PySpark [4] framework, which allows to trivially scale the computation to clusters, for large datasets. The next subchapters will include notes and observations about the PySpark implementation.

Feature Models

The implemented feature models are *Bag Of Words*, *Shingles*, *TF-IDF* and *Embeddings*. All of them are based on PySpark's DataFrames, using native methods from the package `pyspark.sql.functions`. When necessary, Pandas UDF are adopted.

Bag of Words and Shingles

Their implementation is straightforward.

TF-IDF

In PySpark, this model is implemented using `pyspark.ml.features.IDF` and `pyspark.ml.features.CountVectorizer`. A custom implementation was developed but ultimately scrapped: its performance were much worse than the methods provided by the PySpark library.

Embeddings

In this study, a transformer-based model, *Qwen3-embedding-0.6b* [3] quantized to INT8, is used and executed locally using Ollama, chosen for its excellent size to performance ratio.

Unfortunately, the execution environment is unfit for the processing of large batches of embeddings: all experiments are ran on a MacBook Air M1 with 8GB of RAM, shared with user applications and PySpark. This forces to perform benchmarks on (very) small samples.

Naturally, this environment isn't intended for this kind of algorithm. For reference, the Apple Silicon M1 scores around 5 TOPS (Trillion of OPerations per Second) on INT8, while an Nvidia GeForce RTX 5070 computes 246.9 TOPS, about 50 times more than the Apple Silicon M1. Performance are also hit by the low amount of RAM, reaching red levels of RAM usage on MacOS's Activity Manager, further increasing the required runtime.

FastSimHash

SimHash can be naively implemented using an array of counters and iterating over all feature's digests. It's possible to optimize it by using Numba, a Just-In-Time compiler for Python that translates Python and NumPy code into machine code, using SIMD instructions wherever possible. Instead of iterating over all digests, it iterates over byte indexes and uses 8 counters (one for each bit of a byte) to reduce memory updates. This is inspired by Otmar Ertl's FastSimHash implementation [5], which uses a SWAR (SIMD Within A Register) technique instead of pure SIMD instructions. It's a remarkable improvement: performing SimHash with 1000000 digests takes around 14000ms, while FastSimHash takes around 400ms (including compilation time, on a cold boot).

Fingerprint Detection with RDDs

The detection algorithm from the original paper [1] has been replicated in PySpark using RDDs, with some differences. Instead of using a binary search algorithm, an hash map is used: it requires slightly more memory, but it optimizes the candidate search from $O(logn)$ to $O(1)$. The Huffman Code compression strategy is not implemented: it would require a fixed K value, which is not the case for this study. However, it is certainly recommended for production environments where fingerprints are stored, with a fixed K value.

Results

This chapter showcases the benchmark results of the algorithm previously described, across different datasets. The algorithm is ran on a MacBook Air M1 8GB, in a user environment with Safari, Ollama and Visual Studio Code running. PySpark has been configured to use 8 workers.

Unfortunately, the RAM available is too low for this task. MacOS's Activity Monitor memory usage indicator is yellow during execution, reaching red spikes during embeddings computation. This causes severe performance downgrade, which required to lower the sample size for the following analysis. It's expected that a more suitable device with more RAM and/or a dedicated GPU would obtain better performance on large datasets.

Features models: performance

In this section, feature models's performance are tested. As [...] shows, *Shingles* and *Bag of Words* perform similarly, while *TF-IDF* takes much more time. The other models are all local: each data elaboration is independent from other, which makes it efficient and easy to compute across workers. On the other hand, TF-IDF cannot replicate this: it's necessary to create a large, global data structure that counts how many times each word appears in all the dataset, resulting in a slower computation.

Embeddings feature model isn't included, as discussed in the previous chapter.

qqp dataset

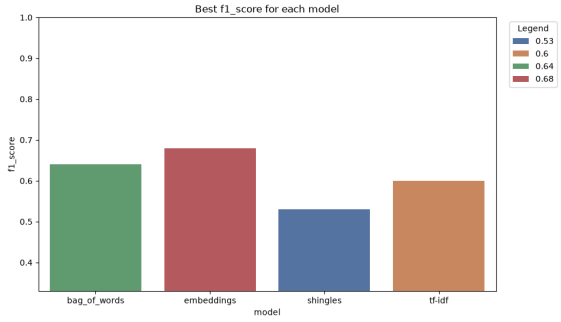


Figure 1: Best F1 score of each feature model

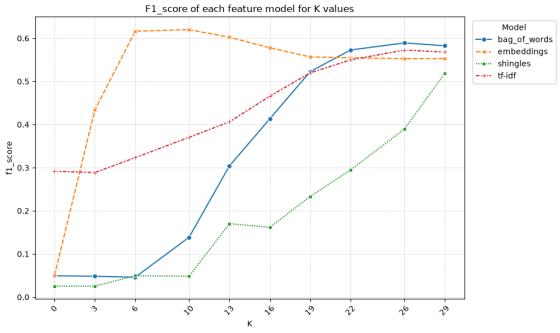


Figure 2: Best F1 score for each K value

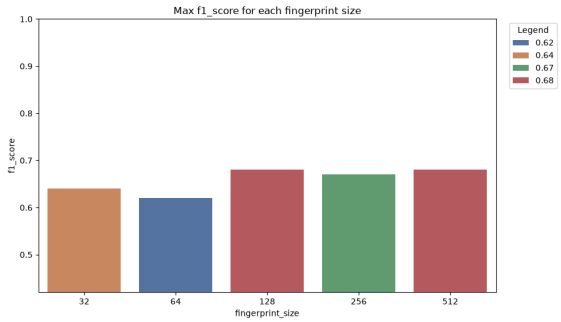


Figure 3: Best F1 score of each fingerprint size

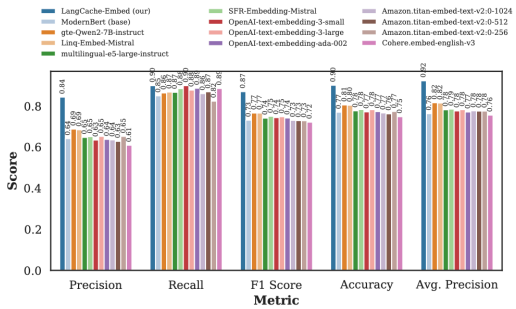


Figure 1: Comparison of embedding-model performance on the Quora dataset. The y-axis shows score, while the x-axis lists metrics. LangCache-Embed (i.e., fine-tuned ModernBERT) exhibits a significant uplift in precision and recall compared to its baseline (non-fine-tuned) version and other state-of-the-art embedding models, highlighting the impact of fine-tuning.

Figure 4: F1 Score of different embedding models, by Gill et al.

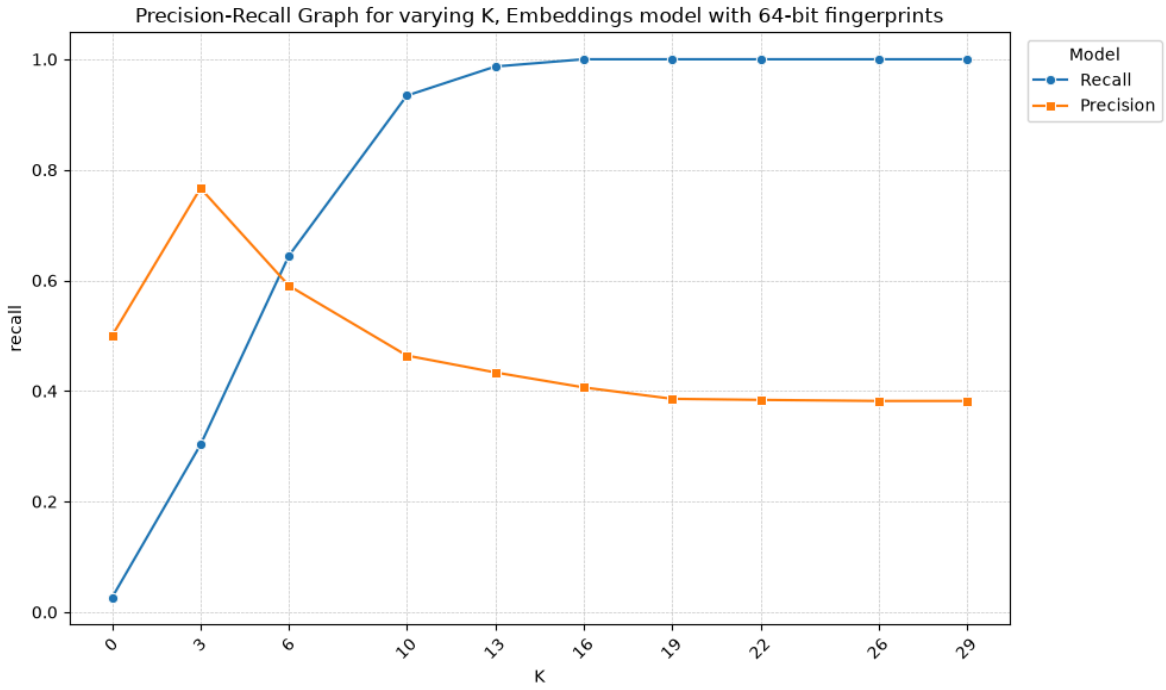


Figure 5: Precision-Recall linegraph

This dataset is the de-facto standard for near-duplicate detection algorithms. It features more than 300,000 rows of text pairs, labelled as "near-duplicate" or "not near-duplicate", which are titles extracted from Quora's questions, a famous Q&A website. A pair is considered "near-duplicate" if its semantic meaning is the same.

In this benchmark, a sample of 199 rows are considered, splitted in 76 near-duplicates and 123 non near-duplicates.

Figure 1 shows feature model's f1-score on the chosen sample: as expected, *Embeddings* is the best performing model, although not by much. Figure 2 shows a clearer image: *Embeddings* is the only model capable of truly detecting duplicates, as shown by the spike around $K = 6$.

However, *Embeddings* score isn't as high as one might expect: it's possible that the heavy quantization applied to the produced weights drags down its score, or the (rather small) embedding model might not be "smart" enough for this task. In order to verify this, the computed f1-score is compared to the result of Gill et al. [6]. Figure 4 shows that larger, state-of-the-art generic-purpose embedding models settle on a f1-score of 0.77, which isn't far from the best obtained score from this work (which is 0.68). Moreover, Gill et al. show that fine-tuning can drastically improve performance: in this work, the embedding model is used as-is. This can be considered for a possible improvement.

From Figure 3 it's possible to assume that larger fingerprint sizes don't affect the f1-score (for this dataset). Most likely, large fingerprint sizes might be useful in huge datasets, where the collision risk is higher.

Considering the Embedding feature model and a fingerprint size of 64 bits, the optimal K value seems to be between 3 to 6, from Figure 5. This guarantees an optimal environment for this algorithm applicaiton.

Wikipedia dataset

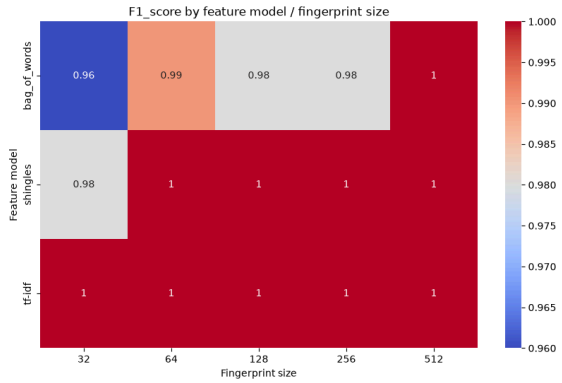


Figure 6: F1 score confusion matrix

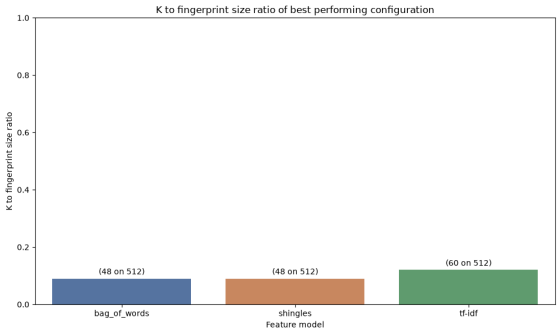


Figure 7: Optimal K to fingerprint size ratio for best performing configurations

It's a dataset of 998 Wikipedia entries. Each row contains two entries that can be syntactically duplicate or not. Duplicate entries are the same entries but modified to introduce noise: about 10% of the words are altered. Meanwhile, non-duplicate entries are completely Wikipedia entries.

Once again, the *Embeddings* feature model isn't represented: some entries are over 87,000 characters long, which immediately saturates the available memory.

Figure 6 shows a promising result: all feature models achieve an almost perfect score. It's also expected that *Embeddings* would perform similarly.

Meanwhile, Figure 7 assures that the optimal K value to fingerprint size ratio is around 0.1, within limits for an optimal execution.

Applications

What follows are some application ideas. This work can be used in all context where near-duplication detection is required, as long as the K value is low enough.

- As the source paper [1] suggests, remove near-duplicate pages to improve web crawling processes efficiency. This can be extended to any data cleanup workflow.
- Given a dataset of known spam, it's possible to filter out incoming spam comparing the incoming mails with the dataset. This can also be used to prevent plagiarism.
- Build advanced file systems, able to bundle together files with similar content.

Future explorations

- The inadequately small amount of RAM made the *Embeddings* model not fit for large datasets. Running it on a more adequate machine (with a dedicated GPU, or with 16GB+ of memory) would surely fare much better performance.
- Despite this work focus on textual data, it's possible to adapt the described algorithm to any other type of media by using a different feature extraction model. For example it's possible to describe programs using their AST, or images with advanced multimodal embedding models.
- This algorithm is usually executed on a data center (or a single server), which requires significant processing power when using embedding models on large batches of data. It's possible to alleviate this by computing embeddings on the client hardware: modern laptops are equipped with TPUs

(Tensor Processing Unit) like Apple's ANE (Apple Neural Engine): dedicated circuit optimized to perform tensor operations, ideal for quick, local embedding computation.

Conclusion

The results have shown that the near-duplicate detection algorithm ideated by Manku et al [1] is consistent with literature. Also, it's flexible enough to support advanced feature models like embeddings, detecting semantic and syntactic near-duplicates for low K values.

Bibliography

[1] A. D. S. Gurmeet Singh Manku Arvind Jain, "Detecting Near-Duplicates for Web Crawling," 2007.

[2] M. S. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," 2002.

[3] Y. Zhang *et al.*, "Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models," *arXiv preprint arXiv:2506.05176*, 2025.

[4] [Online]. Available: https://spark.apache.org/docs/latest/api/python/index.html

[5] O. Ertl, "Speeding up SimHash by 10x using a bit hack," 2023.

[6] W. Gill *et al.*, "Advancing Semantic Caching for LLMs with Domain-Specific Embeddings and Synthetic Data." [Online]. Available: https://arxiv.org/abs/2504.02268